SPRINT 1 · Plano (Análise e Planeamento)

Modelo de Sistematização de Entregáveis

Projeto: ACaminho — aplicação móvel TVDE
Equipa: Grupo K — A caminho | Scrum Master: Simão Cerejo | Desenvolvimento: Gabriel Costa, Gabriel Rocha e Diogo Pereira

Datas do sprint	Data limite de entrega (indicativa)	Ponderação na nota final
Semanas de 12 out. a 08 nov. 2026 (sessões 5 a 8)	**05-11-2026**	**20% (instrumento ii)**

Este documento organiza o objetivo do Sprint 1, os entregáveis a produzir, os critérios de avaliação e os modelos preenchiveis para os documentos de análise e planeamento: User Stories, modelos do sistema, arquitetura e plano dos sprints seguintes.

1. Objetivo do Sprint

Sprint Goal sugerido: *"Ter os requisitos do produto levantados e priorizados como User Stories, os modelos do sistema e a arquitetura definidos, e o trabalho decomposto em tarefas planeadas para os Sprints 2 e 3."*

- **Levantar e escrever requisitos** sob a forma de User Stories, a partir da proposta validada no Sprint 0.
- **Priorizar as User Stories** com o docente no papel de Product Owner, em Sprint Planning.
- **Desenhar os modelos do sistema** (casos de uso, diagramas de classes/componentes ou de dados, consoante o tipo de produto).
- **Definir a arquitetura** (cliente/servidor, base de dados, plataforma desktop/web/móvel).
- **Decompor as User Stories em tarefas técnicas** e estimar o trabalho para os Sprints 2 e 3.
- **Registar pelo menos uma Daily Scrum** (simulada ou registada) para acompanhar o progresso.
- **Realizar Sprint Review e Sprint Retrospective** no final do sprint.

2. Entregável e avaliação

Entregável	Formato	Data limite	Ponderação
(ii) Análise e planeamento: requisitos, User Stories, modelos, arquitetura	Documento(s) + apresentação no Sprint Review	05-11-2026	20%

Penalização por atraso: 10%/20%/40%/80% consoante o número de datas em atraso, nos termos do regime completo descrito na Proposta de Revisão da FUC. Recorda-se que um sprint só pode ser entregue depois de o anterior estar entregue (ou em simultâneo).

Nota de preenchimento: esta checklist é um critério de conclusão, não uma declaração automática de que todos os itens já foram cumpridos. Confirmar cada caixa apenas quando existir evidência (diagrama, README, registo ou apresentação).

3. Definição de "Concluído" do Sprint 1

- ☐ Todas as funcionalidades previstas na proposta do Sprint 0 foram traduzidas em User Stories, com formato consistente.
- ☐ As User Stories foram priorizadas com o Product Owner (MoSCoW ou similar) em Sprint Planning.
- ☐ Os modelos do sistema (casos de uso e/ou diagramas) foram desenhados e refletem as User Stories priorizadas.
- ☐ A arquitetura (camadas, tecnologias, base de dados, plataforma) está definida e documentada.
- ☐ Os requisitos não funcionais relevantes foram identificados e documentados à parte das User Stories.
- ☐ Existem wireframes ou um mapa de navegação para os ecrãs principais.
- ☐ O contrato de API (se aplicável) e o README inicial do repositório foram criados.
- ☐ As User Stories estão decompostas em tarefas técnicas, com estimativa de esforço.
- ☐ Existe um plano indicativo do que será feito nos Sprints 2 e 3.
- ☐ Foi registada pelo menos uma Daily Scrum.
- ☐ Sprint Review (apresentação ao docente) e Sprint Retrospective foram realizados.

Índice de figuras

Figura 1-Menu de registo....8
Figura 2-Menu Motorista....8
Figura 3-Menu Motorista 2....8
Figura 4-Menu Passageiro 2....8
Figura 5-Menu Passageiro....8
Figura 6-Menu Administrador....9
Figura 7-Mapa de Navegação....9
Figura 8- Diagrama Use Case US01....15
Figura 9-Diagrama use case US02....15
Figura 10- Diagrama use case US03....16
Figura 11-Diagrama use case US04....16
Figura 12-Diagrama use case US05....17
Figura 13-Diagrama use case US06....17
Figura 14-Diagrama use case US07....18
Figura 15-Diagrama use case US08....18
Figura 16- Diagrama use case US09....19
Figura 17-Diagrama use case US010....19
Figura 18-Diagrama de Classes....20

Nota metodológica: os modelos e documentos das secções seguintes são a primeira versão do design do sistema ("as designed"). Serão atualizados nos Sprints 2 e 3 à medida que a implementação avançar, para refletir o que foi efetivamente construído ("as built"); as divergências entre o que foi planeado aqui e o que foi implementado devem ser registadas, não escondidas.

4. Sprint Goal e Product/Sprint Backlog

4.1 Sprint Goal

Definir e documentar os requisitos prioritários do ACaminho através de User Stories, modelos do sistema e arquitetura técnica, decompondo o trabalho de implementação e teste para os Sprints 2 e 3, sem confundir funcionalidades já implementadas com funcionalidades ainda planeadas.

4.2 Product Backlog priorizado (User Stories)

#	User Story	Prioridade	Estimativa
US01	Como utilizador, quero criar conta, iniciar sessão e aceder às funcionalidades de acordo com o meu papel (passageiro, motorista ou administrador), para utilizar a aplicação de forma segura.	Must	5 pontos
US02	Como passageiro, quero indicar a origem e o destino e consultar os dados do pedido, para solicitar uma viagem através do ACaminho.	Must	8 pontos
US03	Como motorista, quero consultar os pedidos disponíveis, ver a distância até à recolha e aceitar uma viagem, para selecionar serviços mais vantajosos e prestar os mesmos ao passageiro.	Must	8 pontos
US04	Como passageiro e motorista, quero acompanhar o estado da viagem e consultar os dados relevantes da mesma, para saber em que fase se encontra o serviço.	Must	8 pontos
US05	Como motorista, quero atualizar os dados do veículo associado ao meu perfil, para manter a informação do veículo correta e visível ao passageiro.	Should	3 pontos
US06	Como passageiro e motorista, quero avaliar a outra parte depois de uma viagem concluída, para contribuir para a confiança e qualidade do serviço.	Should	5 pontos
US07	Como motorista, quero consultar indicadores de receita por hora e por quilómetro, registar custos e gerar relatórios PDF, para analisar a rentabilidade do meu trabalho.	Should	8 pontos
US08	Como utilizador, quero submeter um pedido de suporte associado a um problema ou viagem e acompanhar a sua resolução, para obter ajuda quando necessário.	Should	5 pontos
US09	Como motorista, quero visualizar no mapa a localização relevante para a recolha e a rota da viagem, para reduzir confusões no ponto de encontro.	Should	8 pontos
US10	Como administrador, quero gerir utilizadores, motoristas, veículos e viagens, para supervisionar o funcionamento da plataforma.	Could	8 pontos

4.2.1 Critérios de aceitação (exemplos onde se verifica)
Os critérios seguintes concretizam a confirmação das User Stories prioritárias. São propostas para validação no Sprint Planning.
US01 — Autenticação e papel
• Dado um utilizador com credenciais válidas, quando inicia sessão, então acede à aplicação.
• Dado um utilizador autenticado, quando consulta as funcionalidades, então só vê as opções permitidas pelo seu papel.
• Dadas credenciais inválidas, quando tenta iniciar sessão, então a aplicação apresenta um erro e não cria uma sessão.

US02 — Pedido de viagem
• Dado um passageiro autenticado, quando indica origem e destino válidos e confirma o pedido, então o pedido é registado com o estado inicial definido (pedido aceite).
• Se a origem ou o destino estiverem em falta, quando tenta submeter, então a aplicação impede o envio e identifica os campos em falta.
• Se ocorrer uma falha de rede ou falha no pagamento, então a aplicação informa que o pedido não foi confirmado.
US03/US04 — Aceitação e acompanhamento
• Dado um pedido disponível, quando um motorista o aceita, então a viagem fica associada a esse motorista e deixa de estar disponível para aceitação por outros.
• Dado que a viagem está ativa, quando o motorista atualiza o estado para chegada, início ou conclusão, então passageiro e motorista observam o novo estado.
• Antes de executar uma ação crítica, a aplicação solicita uma confirmação explícita; uma ação cancelada não altera o estado da viagem.

4.2.2 Fundamentação da priorização
Must: autenticação, pedido, aceitação e acompanhamento constituem o fluxo mínimo demonstrável do serviço. Should: veículo, avaliações, rentabilidade/relatórios, suporte e mapa melhoram a operação e a experiência, mas podem ser faseados. Could: administração completa é relevante para a gestão da plataforma, mas não bloqueia a demonstração do fluxo principal. A ordenação valor/esforço é indicativa: privilegia primeiro o fluxo passageiro–motorista e deixa funcionalidades de gestão e análise para interações posteriores.
As estimativas em pontos e a classificação MoSCoW são propostas de planeamento; devem ser confirmadas pela equipa e pelo docente. O feedback recolhido junto de um motorista foi usado como input exploratório para considerar distância até à recolha, ações críticas, rentabilidade, relatórios e suporte; não constitui uma amostra representativa de todos os motoristas.

4.3 Requisitos não funcionais

#	Requisito não funcional	Categoria	User Story(ies) associada(s)
NF01	As alterações de estado das viagens devem ser persistidas de forma consistente; a aceitação não pode atribuir a mesma viagem a dois motoristas em simultâneo.	Consistência / desempenho	US03, US04
NF02	Os dados e operações devem respeitar autenticação e autorização por papel; um utilizador só pode consultar ou alterar os dados que lhe dizem respeito. As regras Firestore devem ser testadas antes da entrega.	Segurança / privacidade	US01–US10
NF03	Os ecrãs devem apresentar estados e ações de forma clara, com confirmação para operações críticas e navegação adequada a um telemóvel.	Usabilidade	US02–US06, US08–US09
NF04	A localização deve ser pedida com consentimento e usada apenas para as funções necessárias; não se deve assumir disponibilidade contínua nem precisão absoluta do GPS.	Privacidade / RGPD	US02, US04, US09
NF05	A aplicação deve manter separação entre interface, lógica de apresentação e acesso a dados, para facilitar manutenção e testes.	Manutenibilidade	US01–US10
NF06	A aplicação deve comunicar falhas de rede e erros de operações sem indicar falsamente que uma viagem foi criada, aceite ou concluída.	Robustez	US02–US04, US08

5. Modelos do sistema

5.1 Casos de uso

Ator	Use Case	User story Referente	Diagrama
Utilizador	Criar conta, iniciar sessão e aceder às funcionalidades de acordo com o seu papel	US01	1
Passageiro	Indicar a origem e o destino e consultar os dados do pedido, para solicitar uma viagem	US02	2
Motorista	Consultar os pedidos disponíveis, ver a distância até à recolha e aceitar uma viagem	US03	3
Motorista e Passageiro	Quero acompanhar o estado da viagem e consultar os dados relevantes da mesma	US04	4
Motorista	Atualizar os dados do veículo associado ao seu perfil	US05	5
Motorista e Passageiro	Avaliar a outra parte depois de uma viagem concluída	US06	6
Motorista	Consultar indicadores de receita por hora e por quilómetro, registar custos e gerar relatórios PDF	US07	7
Utilizador	Submeter um pedido de suporte associado a um problema ou viagem e acompanhar a sua resolução	US08	8
Motorista	Quero visualizar no mapa a localização relevante para a recolha e a rota da viagem	US09	9
Administrador	Gerir utilizadores, motoristas, veículos e viagens	US10	10

5.2 Modelo de dados / diagrama de classes ou componentes

- **EstadoViagem**: (Pedida, Aceite, Motorista_Chegou, Em_curso, Concluída, Cancelada)
- **Passageiro**: (id_passageiro: int, nome: String, email : String, dados_perfil : String)
- **Motorista**: (id_motorista : int, nome: String, email: String, disponibilidade : String, classificacao : float, dados_perfil : String)
- **Administrador**: (id_admin : int, nome : String, email : String)
- **Veículo**: (id_veiculo : int, id_motorista : int, marca : String, modelo : String, ano : int, matricula : String, url_foto : String)
- **Viagem**: (id_viagem : int, id_passageiro : int, id_motorista : int, origem : String, destino : String, estado : EstadoViagem, preco : float, data_criacao : Timestamp)
- **AvaliaçãoViagem**: (id_avaliacao : int, id_viagem : int, id_utilizador_origem : int, id_utilizador_destino : int, pontuacao : int, comentario : String, criado_em : Timestamp)
- **PedidoSuporte**: (id_suporte : int, id_utilizador : int, id_viagem : int, categoria : String, descricao : String, estado : String, data_pedido : Timestamp)
- **DespesaMotorista**: (id_despesa : int, id_motorista : int, valor : float, data_despesa : Timestamp)
- **RelatórioMotorista**: (id_relatorio : int, id_motorista : int, periodo_inicio : Timestamp, periodo_fim : Timestamp, receita : float, custos : float, resultado : float)

Associações:

- Passageiro -Viagem (1 para 0..N), (Um passageiro pode solicitar várias viagens; cada viagem pertence a 1 passageiro);
- Motorista -Viagem (0..1 para 0..N), (Uma viagem começa com 0 motoristas e fica com 1 quando aceite; um motorista pode conduzir várias viagens);
- Motorista -Veículo (1 para 0..N), (Um motorista pode ter vários veículos registados);
- Viagem - AvaliaçãoViagem (1 para 0..2), (Uma viagem concluída pode ter até 2 avaliações, uma de cada interveniente);
- Passageiro - PedidoSuporte (1 para 0..N), (Um passageiro pode submeter vários pedidos de suporte);
- Motorista - PedidoSuporte (1 para 0..N), (Um motorista pode submeter vários pedidos de suporte);
- PedidoSuporte - Viagem (0..1 para 1), (Um pedido de suporte pode opcionalmente estar associado a 1 viagem);
- Motorista - DespesaMotorista (1 para 0..N), (Um motorista pode registar várias despesas ao longo da atividade);
- Motorista - RelatórioMotorista (1 para 0..N), (Um motorista pode gerar e consultar vários relatórios de desempenho e receita);
- Administrador – PedidoSuporte(1 para 0..N),(O administrador gere os pedidos de suporte ou não consoante a sua existência).

5.3 Wireframes e mapa de navegação



Figura 1-Menu de registo

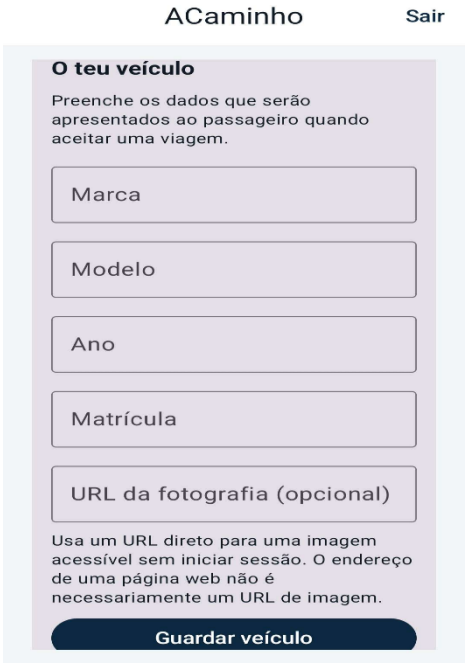


Figura 2-Menu Motorista



Figura 3-Menu Motorista 2

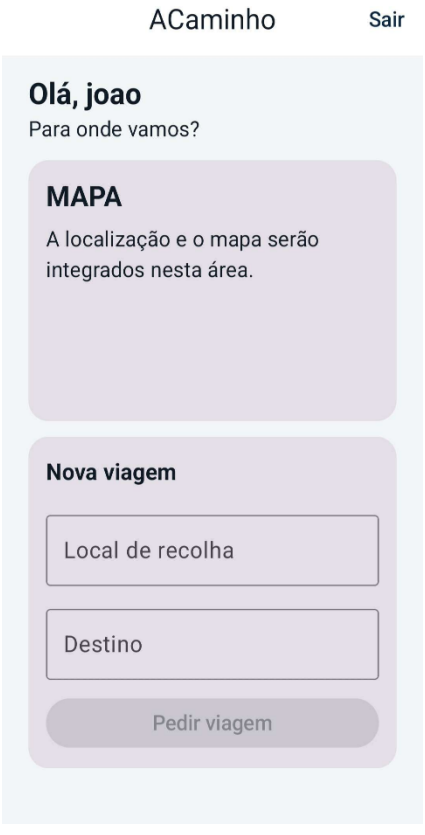


Figura 5-Menu Passageiro

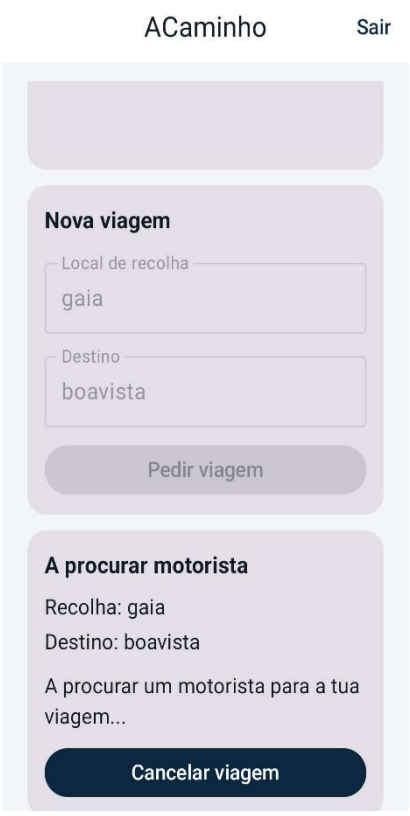


Figura 4-Menu Passageiro 2

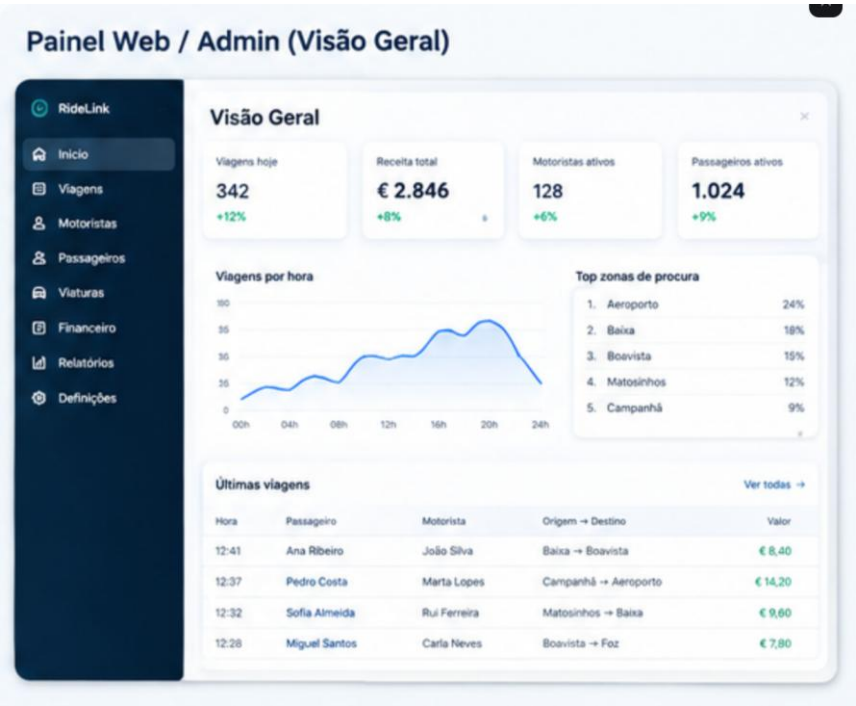


Figura 6-Menu Administrador

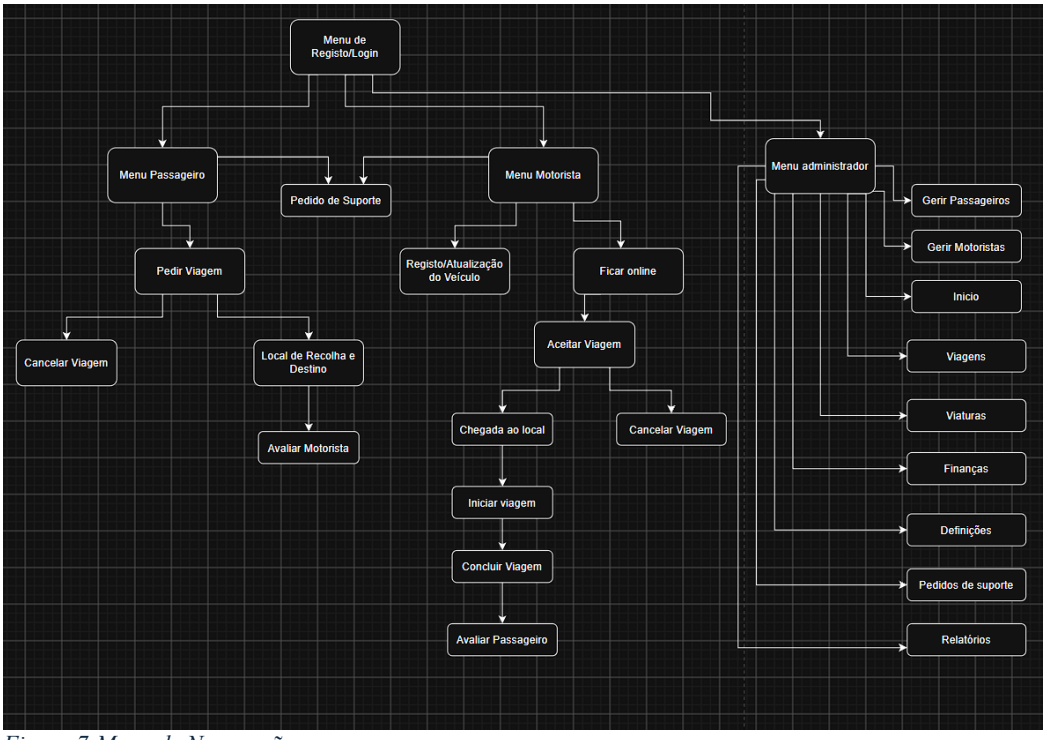


Figura 7-Mapa de Navegação

6. Arquitetura do sistema

6.1 Camadas e tecnologias

Camada / Componente	Tecnologia escolhida	Justificação breve
Frontend / interface	Android nativo: Kotlin + Jetpack Compose	Compatível com o produto móvel e permite construir ecrãs declarativos; já utilizado no projeto.
Backend / lógica funcional	Firebase Authentication + Cloud Firestore; ViewModels e Repositories no cliente	Authentication trata identidade; Firestore fornece persistência e atualizações em tempo real. A lógica de apresentação é separada por ViewModels/Repositories. Não existe uma API REST própria confirmada.
Base de dados	Cloud Firestore (NoSQL)	Integração com Firebase e suporte a listeners em tempo real para estados e pedidos de viagem.
Plataforma de implantação (desktop/web/móvel)	Aplicação móvel Android	O público-alvo (passageiros e motoristas) utiliza a aplicação em telemóveis Android.
Controlo de versões e colaboração	Git + GitHub	Histórico de alterações, colaboração e repositório do projeto ACaminho.

6.2 Contrato de API (se cliente/servidor)

Endpoint	Método	Parâmetros / corpo	Resposta esperada
Não se aplica neste contexto	—	A aplicação acede diretamente ao Firebase Authentication e ao Cloud Firestore através dos SDKs Android; não foi identificada uma API REST própria.	Operações do SDK devolvem resultado/erro; listeners Firestore atualizam os dados observados.
Operação lógica: aceitar viagem	Transação Firestore (não é endpoint HTTP)	tripId, driverId; validar que a viagem está REQUESTED e que o motorista está ONLINE; atualizar viagem e estado do motorista de forma atómica.	Sucesso se a transação cumprir as condições; erro/conflito se outro motorista já aceitou ou o estado mudou.

6.3 Documentação de código (README inicial)

- ☐ Existe um README no repositório com o nome e objetivo do projeto.
- ☐ O README descreve a estrutura de pastas/módulos prevista.
- ☐ O README indica como instalar dependências e correr o projeto localmente (mesmo que ainda incompleto).

7. Decomposição de tarefas e plano dos próximos sprints

7.1 Tarefas técnicas por User Story

User Story	Tarefa técnica	Sprint previsto	Estimativa
US02	Desenhar e implementar ecrã do passageiro com origem/destino e validação dos campos.	Sprint 2	3 h
US02	Definir estrutura de dados do pedido de viagem e persistir pedido no Firestore.	Sprint 2	3 h
US09	Avaliar fornecedor de mapas, configurar chave/SDK e apresentar mapa e localização com permissões explícitas.	Sprint 2	4 h
US03	Implementar lista de pedidos disponíveis em tempo real, apresentar distância até à recolha e permitir filtro de distância máxima.	Sprint 2	4 h
US03	Implementar aceitação segura da viagem, validando estado e disponibilidade do motorista; incluir confirmação deliberada da ação.	Sprint 2	4 h
US04	Implementar estados da viagem e apresentação de estado/dados entre passageiro e motorista, com confirmação nas ações críticas.	Sprint 2	5 h
US05	Implementar edição e validação dos dados do veículo do motorista.	Sprint 2	3 h
US06	Implementar submissão e consulta de avaliações após conclusão.	Sprint 2	3 h
US07	Definir modelo de custos e cálculo de €/h e €/km, distinguindo valores estimados/reais, e gerar PDF por período.	Sprint 3	6 h
US08	Criar formulário de suporte, registo do pedido e estados de acompanhamento.	Sprint 3	4 h
US10	Criar vistas de administração e aplicar regras de autorização por papel.	Sprint 3	6 h

7.2 Plano indicativo dos Sprints 2 e 3

Sprint 2 — Codificação e testes: implementar o fluxo principal do passageiro e do motorista (US02–US05), começar pelo pedido de viagem e pelo mapa/localização, concluir a sincronização dos estados, reforçar as ações críticas e testar as transições e permissões. Implementar avaliações (US06) se a capacidade do sprint permitir.

Sprint 3 — Refinamento, implantação e validação: corrigir problemas encontrados, validar regras de segurança e recuperação de viagens, realizar testes com utilizadores, melhorar a experiência de mapa/localização e preparar rentabilidade e relatórios (US07–US08), suporte (US09) e administração (US12) conforme o tempo disponível. O plano é indicativo e deve ser revisto com o Product Owner.

8. Registo de Daily Scrum

Data	O que foi feito ontem	O que será feito hoje	Obstáculos
20/10/2026	Análise em equipa ao conteúdo a melhorar no Sprint 0.	Correção do conteúdo do Sprint 0, com base na avaliação do docente com uma reunião de equipa.	Dificuldades em encontrar diferenças entre aplicações concorrentes devido à falta de informação.
23/10/2026	Análise ao conteúdo da aula do dia 16/10 e definição de User Stories.	Inserção das User Stories no relatório.	Confirmar prioridades e critérios de aceitação; verificar se todos os requisitos da proposta inicial estão representados.
27/10/2026	Análise ao conteúdo da aula do dia 23/10	Realização dos diagramas de acordo com as user stories, divisão de tarefas para agilizar o trabalho a ser realizado.	Dificuldades em fazer as associações corretas no diagrama de classes, que elementos colocar.
30/10/2026	Continuação dos Diagramas	O Scrum Master (Simão Cerejo), sugeriu alterações nos diagramas de classes, aceites pela equipa e também sugeriu uma nova US.	Definição das associações e elementos nos diagramas.
05/11/2026	Análise final ao relatório, correção de erros ortográficos e verificação da estrutura.	Correção de erros e também melhorias na organização do relatório.	Perceber como ficaria melhor explicado o diagrama de classes, com texto e imagem ou só imagem.

9. Sprint Review e Sprint Retrospective

9.1 Notas do Sprint Review (apresentação ao docente)

A preencher após a apresentação real ao docente. Registar: artefactos apresentados (Product Backlog, modelos, arquitetura, plano de sprints), comentários concretos do Product Owner/docente, alterações pedidas e decisões tomadas. Não foi fornecido feedback efetivo do docente, pelo que não deve ser inventado.

9.2 Sprint Retrospective

O que correu bem	O que correu mal	Ações para o próximo sprint
—	—	—

Anexo A — Registo de Decisão de Arquitetura (ADR-001)
Decisão: aplicação Android nativa com Kotlin, Jetpack Compose e Firebase
Estado: Aceite para a versão académica atual; rever se surgirem limitações técnicas.
Contexto: O ACaminho é um protótipo móvel TVDE que necessita de autenticação, persistência de perfis/viagens e atualização do estado das viagens. O tempo de desenvolvimento do semestre é limitado.
Decisão: Utilizar Kotlin e Jetpack Compose na aplicação Android, organizar o código com ViewModels e Repositories e utilizar Firebase Authentication e Cloud Firestore para autenticação e persistência. Não introduzir microserviços nem uma API REST própria enquanto não existir uma necessidade concreta.
Consequências positivas: Reduz o trabalho de infraestrutura; permite autenticação e atualizações em tempo real; mantém a implementação dentro das tecnologias já usadas no projeto.
Consequências e riscos: Existe dependência do ecossistema Firebase; as regras Firestore têm de ser rigorosamente configuradas e testadas. Mapas exigem escolher um fornecedor, configurar credenciais e verificar condições/custos. O modelo de domínio deve ser atualizado à medida que o código real evoluir.

10. Anexos

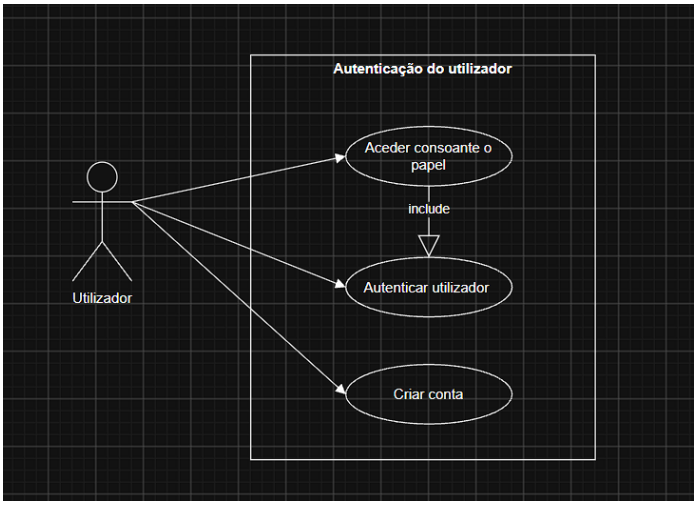


Figura 8- Diagrama Use Case US01

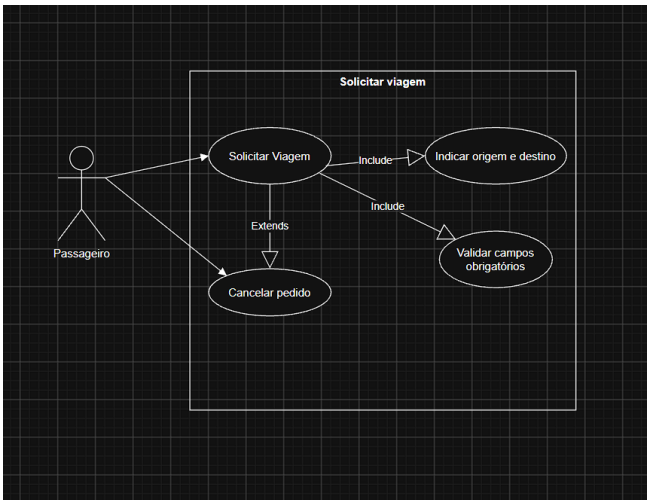


Figura 9-Diagrama use case US02

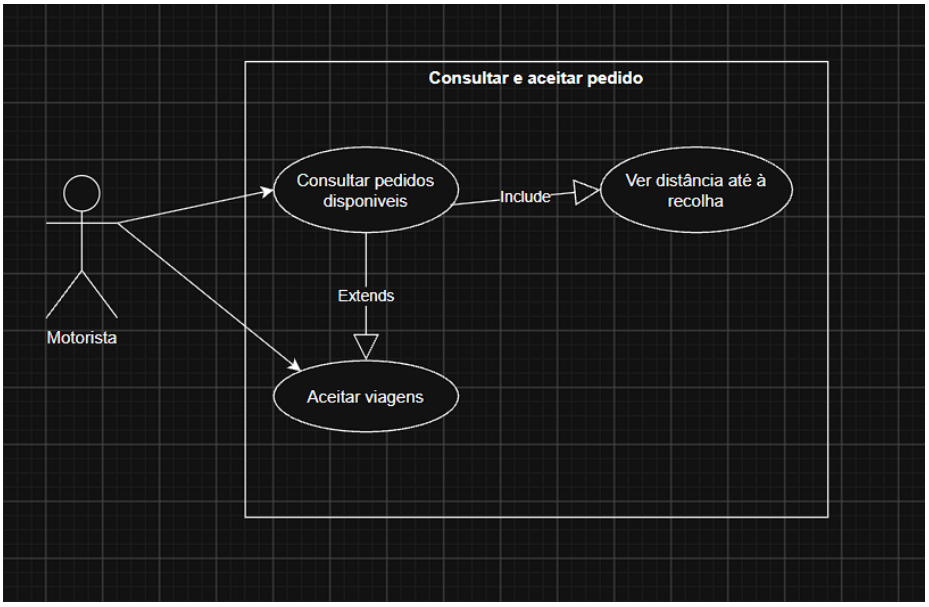


Figura 10- Diagrama use case US03

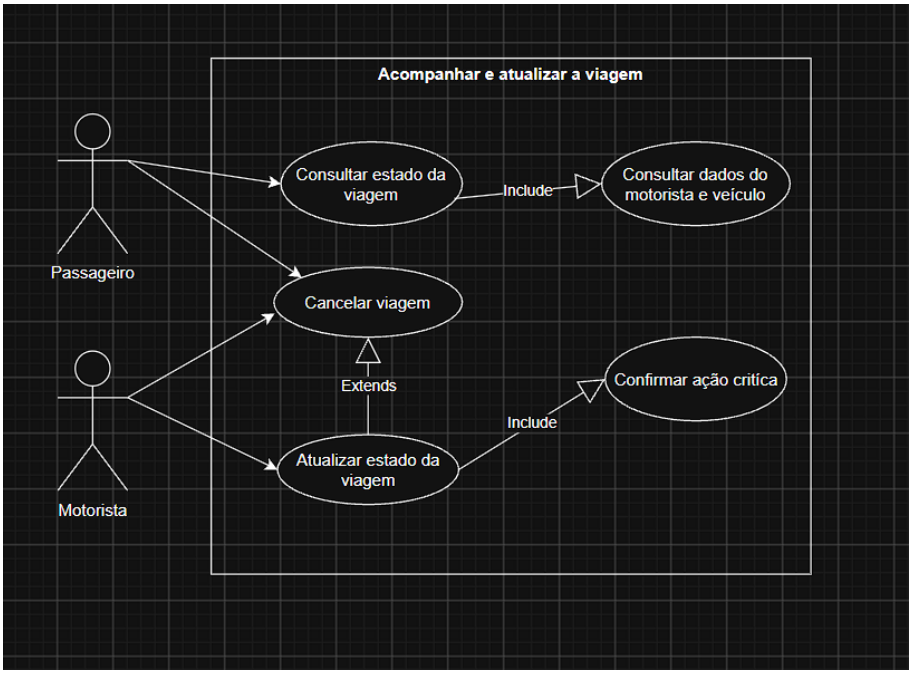


Figura 11-Diagrama use case US04

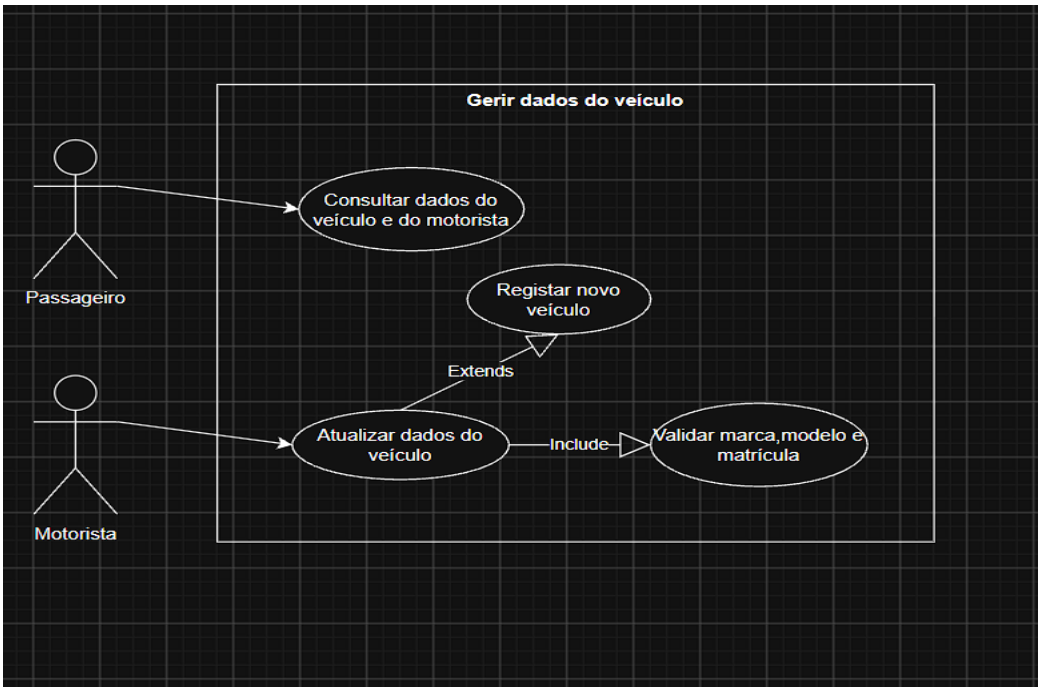


Figura 12-Diagrama use case US05

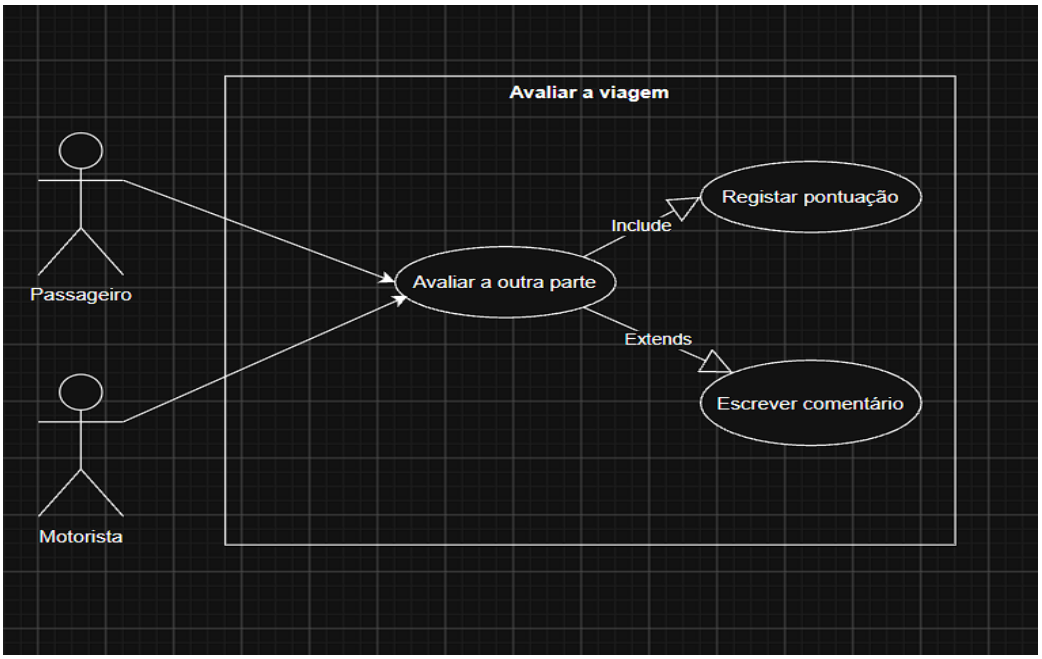


Figura 13-Diagrama use case US06

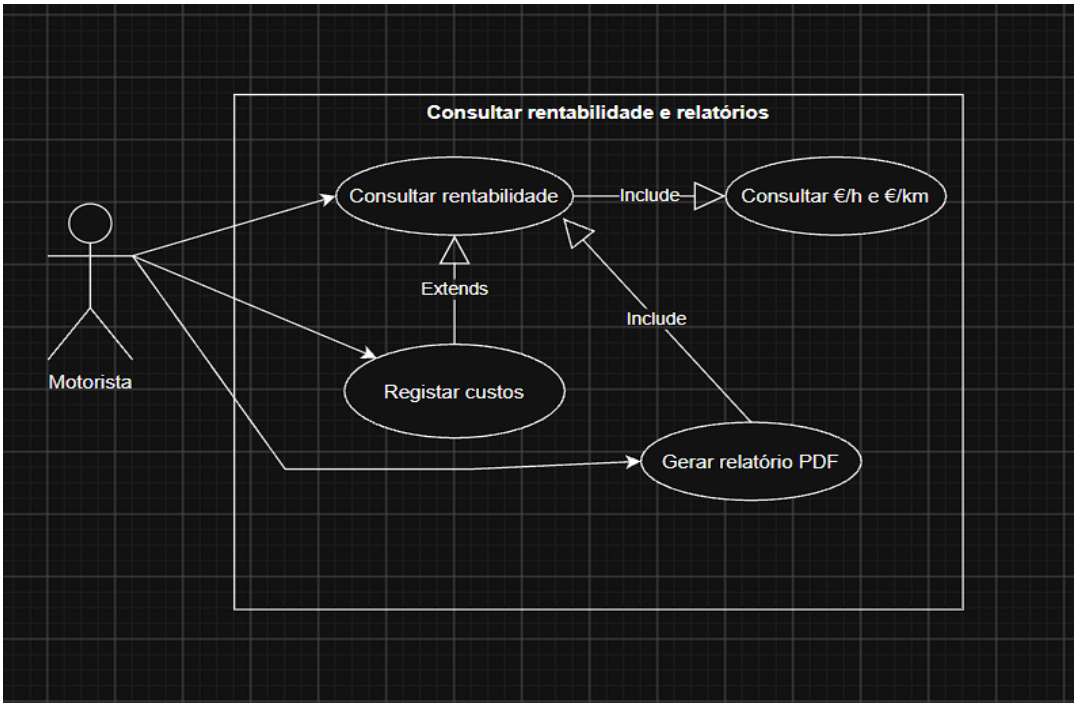


Figura 14-Diagrama use case US07

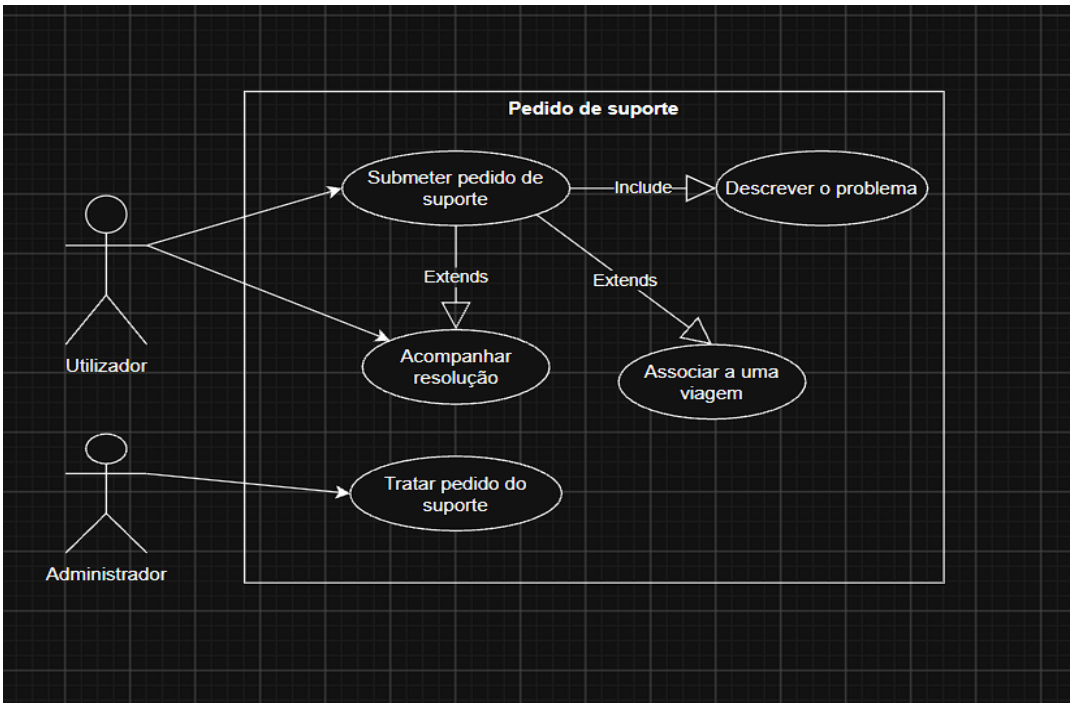


Figura 15-Diagrama use case US08

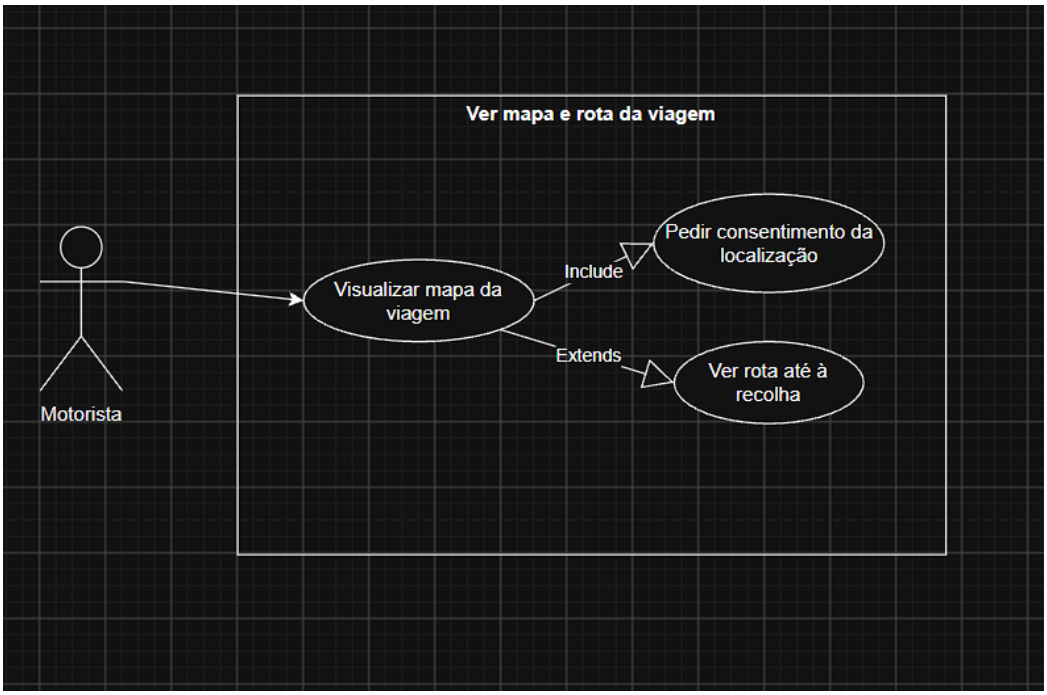


Figura 16- Diagrama use case US09

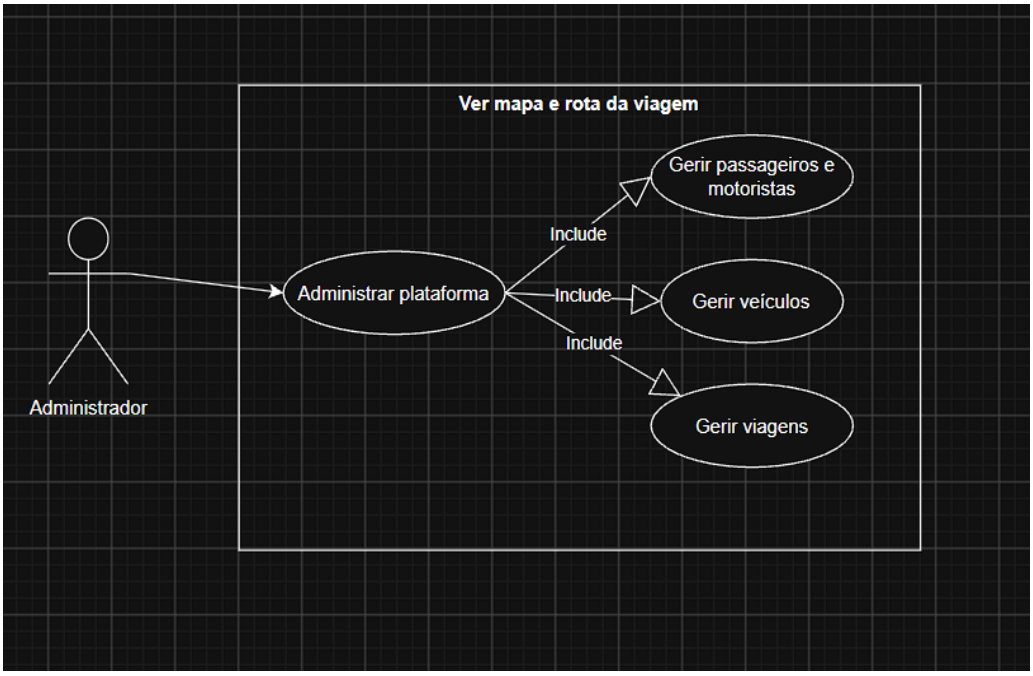


Figura 17-Diagrama use case US010

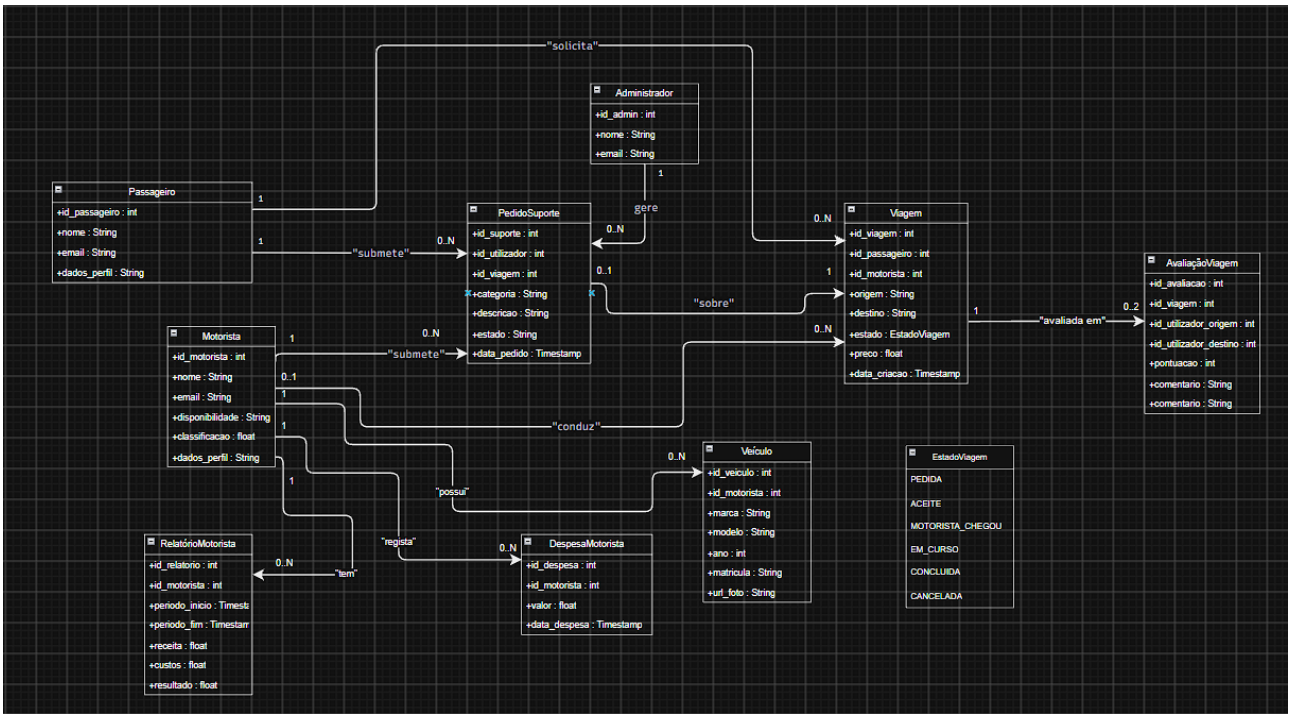


Figura 18-Diagrama de Classes